

A Attractors estimation

In all of our experiments, we estimate the concept’s Attractor as the mean value of the hidden state of all samples (e.g., all toxicity samples or all author questions) for the Attractors’ layer. The layer is chosen as the one that maximizes the inter-distances while minimizing the intra ones. More specifically, assume $M * N$ prompts, where each set of M prompts corresponds to a specific concept (e.g., Harry Potter). For each layer l , we calculate the concept’s Attractor as

$$\mathbf{a}_i^{(l)} = \frac{1}{M} \sum_{j=0}^{M-1} \mathbf{h}_{i*M+j}^{(l)}, \quad \forall i \in [1, N] \quad (4)$$

where $\mathbf{h}_k^{(l)}$ corresponds to the hidden state of prompt k at layer l . Based on the estimated Attractors, we calculate two different metrics: (1) inter-distance, and (2) intra-distance. The inter-distance is defined as the average distance between all the different pairs of Attractors, i.e.,

$$\text{inter}^{(l)} = \frac{1}{\frac{M^2-M}{2}} \sum_{i=1}^M \sum_{j=i+1}^M \mathcal{D}(\mathbf{a}_i^{(l)}, \mathbf{a}_j^{(l)}) \quad (5)$$

while the intra-distance is calculated as the average distance of each hidden state for its corresponding Attractor (i.e., the spread):

$$\text{intra}^{(l)} = \frac{1}{M * N} \sum_{i=1}^N \sum_{j=0}^{M-1} \mathcal{D}(\mathbf{a}_i^{(l)}, \mathbf{h}_{i*M+j}^{(l)}) \quad (6)$$

where $\mathcal{D}$ is an appropriately chosen distance measure (cosine similarity in our experiments). Based on these measurements, we define Separation as

$$\text{Separation}^{(l)} = \text{inter}^{(l)} - \text{intra}^{(l)} \quad (7)$$

and chose as the ideal layer l^* the layer that maximizes this quantity:

$$l^* = \arg \max_l \text{Separation}^{(l)} \quad (8)$$

In practice, we observe, as expected by the IFS theory, the layer choice is quite clear even by simply examining the accompanying t-sne/distance plots (e.g., Figures 4 and 5). In Figure 13 we depict the Separation we obtain for each forget set, results that agree with Figure 5.

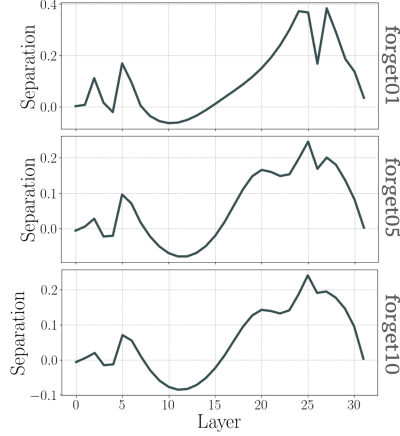


Figure 13: Separation per layer for all three forget sets of TOFU. Interestingly, the optimal layer does not change between different forget sets, strengthening the IFS assumption that a single layer is “responsible” for all authors’ Attractors.

B Attractors across model sizes

In Figure 4 we demonstrated that contemporary LLMs, like LLaMA3.1 [2] and Qwen2.5 [11], form attractors in different layers for many and diverse concepts, from mathematical operations, to literature and programming languages. However, our experiments were based on a specific size (7-8B) casting doubts on the generality of the results. In Figure 14 we demonstrate the IFS nature for a widecollections of LLM sizes, spanning from 3B to 14B models. Our initial observations hold in this expanded family of models too, showcasing that our approach can be used in practice for more or less powerful LLMs, depending on someone’s needs and capabilities.

C Attractor for concept detection

Data. The TOFU benchmark [23] uses a synthetic dataset crafted to test how well LLMs can forget specific information. It features 200 made-up author profiles, each with 20 question-answer

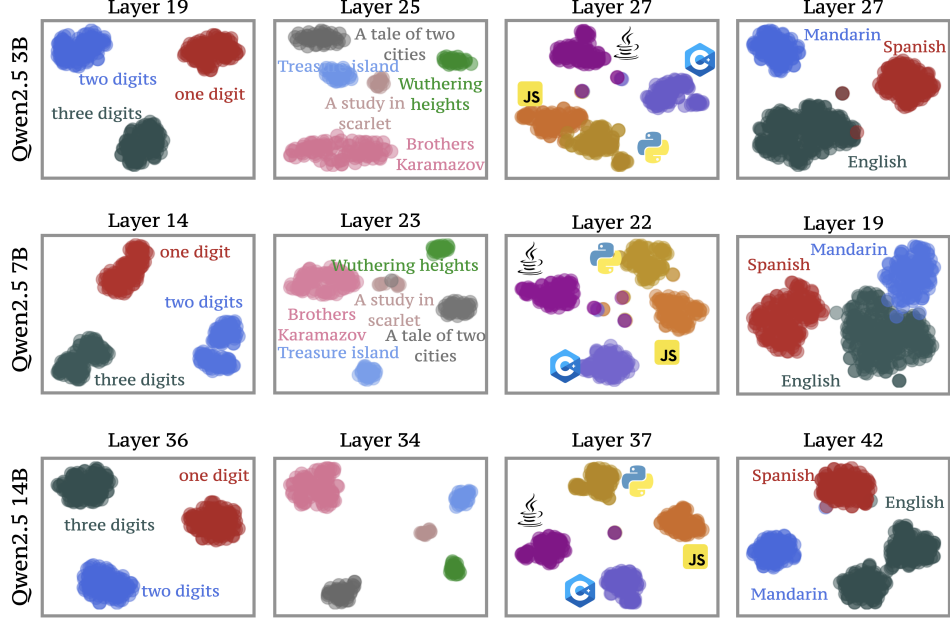


Figure 14: Attractors in Qwen2.5 Qwen2.5-7B [20, 11] family of models, from 3B to 14B. Similarly to Figure 4, the formation of attractors is aparent on all models, although on different layers for each model.

pairs detailing aspects like birthplace, genre, and awards. These profiles were generated using GPT-4, ensuring they don’t exist in any real-world data. To evaluate unlearning, a subset of these profiles, called the “forget set”, is designated for the model to forget. Three different variations were introduced –forget01, forget05, and forget10– that correspond to different percentages of the authors to be forgotten. The rest form the “retain set” which the model should remember. Additionally, TOFU includes evaluation datasets with real authors and general world facts to assess whether unlearning specific information affects the model’s broader knowledge.

Models. Using the fictitious data from above, TOFU then finetuned multiple LLMs on different subsets of them. One was trained on everything but forget10, one in everything but forget05, one in everything but forget01, and finally one was trained on the whole dataset. The fully-trained model is the one used to test different unlearning methods, while the three partially-trained models correspond to the ideal models and parameters (θ^*) that unlearning methods seek.

Evaluation metrics.

1. **Probability:** The Probability metric assesses the model’s confidence in generating the correct answer a given a question q . To normalize for answer length, the probability is adjusted as follows:

$$P(a | q)^{1/|a|} \quad (9)$$

where $|a|$ denotes the number of tokens in the answer. This normalization ensures fair comparison across answers of varying lengths.

2. **ROUGE-L Recall Score:** The ROUGE-L Recall Score measures the overlap between the model’s generated answer and the ground truth answer, focusing on the longest common subsequence. It captures the model’s ability to produce answers that are similar in content and structure to the expected responses, even if the wording differs.
3. **Truth Ratio** The Truth Ratio compares the model’s confidence in a paraphrased correct answer $\tilde{a}$ to its confidence in several perturbed (incorrect) versions $\hat{a} \in A_{\text{pert}}$. It is defined as:

$$R_{\text{truth}} = \min \left(\frac{1}{|A_{\text{pert}}|} \sum_{\hat{a} \in A_{\text{pert}}} \frac{P(\hat{a} | q)^{1/|\hat{a}|}}{P(\tilde{a} | q)^{1/|\tilde{a}|}}, \frac{P(\tilde{a} | q)^{1/|\tilde{a}|}}{\frac{1}{|A_{\text{pert}}|} \sum_{\hat{a} \in A_{\text{pert}}} P(\hat{a} | q)^{1/|\hat{a}|}} \right) \quad (10)$$

This metric reflects the model’s ability to distinguish correct answers from incorrect ones. A lower Truth Ratio indicates better unlearning performance.

4. **Model Utility:** The utility score of the model is derived from the harmonic mean of nine individual measures: answer probability, truth ratio, and ROUGE recall for each of the three evaluation subsets –retain, real authors, and world facts. A higher utility score is indicative of better model performance.
5. **Forget Cutoff:** This metric is introduced by us, and it is depicted in Figure 6 (left). Since our method is about guardrailing specific authors, we are interested in the percentage of author-related questions that are correctly detected (and cutted off).

Baselines. The complete details on all baselines can be found in [38].

D Attractors for traversals

D.1 Drifting away from the toxicity Attractor

Data. The ParaDetox dataset is a key resource for training models to rephrase toxic language into neutral expressions [46]. It comprises over 10,000 English sentence pairs, each featuring a toxic sentence and its non-toxic paraphrase. The dataset was created through a structured crowdsourcing process on Toloka.ai, involving paraphrasing, content preservation checks, and toxicity verification. This approach ensured high-quality data for developing effective detoxification models.

Baselines. In Figure 8 we compared our method against 3 different baselines. Here is a breakdown of each one:

1. **ICL:** ICL, which stands for In-Context Learning [13], utilizes the LLM with specific prompts and a few examples (demonstrations) to guide detoxification without altering model weights.
2. **LoRA:** LoRA, which stands for Low Rank Adaptation [75], finetunes the model on the specific dataset (ParaDetox [46]). Although the “heaviest” of all methods, since it evolves training (some) of the LLM’s parameters, the results are not better than more lightweight approaches, like ours.
3. **ICV:** ICV, which stands for In-Context Vectors [40], calculates a “task vector” using a small set of (paired) in-context examples. This vector encapsulates the task’s essence and is used to modulate the model’s behavior for detoxification tasks without additional fine-tuning.

D.2 Switching language Attractors on the fly

Data. To estimate the programming languages Attractors we used solutions from LeetCode’s problems from <https://huggingface.co/datasets/greengr0ng/leetcode>. Each sample of the dataset consists of the question and its difficulty, as well as the corresponding solutions in Python, Java, C++, and Javascript.

Baselines. In Figure 9 we compared our method against 5 different baselines. Here is the details of each one:

1. **ICV:** ICV, which stands for In-Context Vectors [40], calculates a “task vector” using a small set of (paired) in-context examples. In [41] it can be also found as “PCA”.
2. **Logistic Regression:** A linear classifier applied to the activations of a single layer within the LLM. It serves as a baseline in [41] to assess the effectiveness of simple linear decision boundaries in detecting specific concepts.
3. **Linear Regression:** Similar to logistic regression, the underlying classifier in this case is linear regression.
4. **Difference of Means (DM):** A method that involves directly matching the hidden representations corresponding to specific concepts without any learned transformation.